AMRITA VISHWA VIDYAPEETHAM
SCHOOL OF COMPUTING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Autonomous First-Response Crime Scene Exploration Using a Lightweight Onboard Intelligence System

23CSE399 – Project Phase 1 (Panel Review 2)
March 2026

S.No	Register Number	Student Name	Role
1	CB.SC.U4CSE23062	Karthik Raju	Team Member
2	CB.SC.U4CSE23063	Maheswar Raju	Team Member
3	CB.SC.U4CSE23212	Vijay Krishna Burra	Team Member
4	CB.SC.U4CSE23310	Dhinesh Chandra Boddupally	Team Member
5	CB.SC.U4CSE23421	Kuchipudi Yokshith Sai Ram	Team Member

Project Type: Type 2 – Implementation / Application Project
Guide: Mr. Rajeshwar Yadav, Assistant Professor, Department of CSE

---

1 Project Proposal and Objectives

1.1 Project Overview

This project, titled *Autonomous First-Response Crime Scene Exploration Using a Lightweight Onboard Intelligence System*, is a Type 2 Application Project that addresses a critical safety gap in emergency response operations. The core proposition is straightforward: instead of sending human officers into an unknown, potentially hostile environment as the first act of response, an autonomous mobile robot enters first, maps the environment, identifies regions of interest, and returns with secured evidence — all without any cloud connectivity or GPU hardware.

The system is built on three integrated pillars: SLAM-based autonomous navigation, a fine-tuned Mobile Mamba State-Space Model for lightweight onboard inference, and a forensic-grade data handling pipeline. Together, these components form a complete situational intelligence system designed for deployment in post-incident, pre-entry scenarios where human entry is unsafe.

1.2 Project Objectives

- Develop an autonomous mobile robotic platform capable of navigating unknown indoor environments without human teleoperation.
- Implement SLAM-based frontier exploration to achieve systematic, high-coverage mapping of the environment.
- Fine-tune the Mobile Mamba State-Space Model on an indoor anomaly dataset for real-time Region of Interest (ROI) detection.
- Deploy the complete inference pipeline on a Raspberry Pi, achieving inference latency under 150ms without GPU or cloud.
- Design a forensic data handling pipeline with SHA-256 integrity hashing and AES-256 encryption for evidence preservation.

- Integrate all system modules into a mission-capable simulation framework demonstrating autonomous exploration, SLAM-based navigation, ROI detection using the fine-tuned Mobile Mamba model, and structured data logging and synchronization when connectivity is available.

1.3 Use Case — Hostage Building Scenario

The primary deployment scenario is a post-incident hostage situation. Hostages are trapped inside a building. Attackers have destroyed entry infrastructure, disabled or hacked existing cameras, and may be using RF jammers to prevent communication. Human entry is unsafe prior to intelligence gathering.

In this scenario, the robot is deployed from a safe perimeter and enters the building through an available physical access point — an open door, a window, or a passage through debris. It autonomously explores the interior, identifies suspicious regions (unusual object arrangements, fallen persons, hidden items), logs all data with cryptographic integrity, and returns to its starting position. If Wi-Fi connectivity is restored at any point during or after the mission, the encrypted evidence synchronises to a command server. If not, the data is retrieved when the robot returns.

Key acknowledged limitations of this deployment model: the robot cannot enter if no physical access point exists, and mid-mission battery depletion may prevent its return. Both limitations are documented with mitigation strategies in Section 7.

2 Introduction and Background

2.1 Domain and Problem Context

This project operates at the intersection of four technical domains: Autonomous Mobile Robotics, Lightweight Edge AI, Public Safety and Law Enforcement Systems, and Secure Embedded Computing. The problem it addresses is both practical and underserved in current literature: first responders routinely enter unknown environments without any prior spatial or situational intelligence, placing themselves at significant risk.

Existing crime scene and surveillance systems offer no solution to this problem. They are architecturally designed around static CCTV infrastructure with fixed viewpoints, cloud-dependent inference pipelines, and GPU-based deep learning models. None of these systems can autonomously explore an environment, reposition themselves, or operate without network connectivity. They are tools for retrospective analysis, not proactive first-response safety.

2.2 Motivation

The motivation for this project arose from a specific question: what if the first entity to enter a crime or hostage scene were not a human officer, but an autonomous robot that could gather intelligence and return safely? This framing — referred to throughout as the *first-response exploration paradigm* — is fundamentally different from anomaly detection. The system is not being built to classify whether a crime occurred. It is being built to help responders understand what they are walking into before they walk into it.

This reframing drives every architectural decision in the project. The robot must be mobile, autonomous, and computationally self-sufficient. The inference model must be lightweight enough to run on embedded hardware. The data pipeline must be forensically sound. And the entire system must function without any external network dependency.

2.3 Literature Review

Nine research papers were surveyed as part of this project, spanning CVPR 2024, AAAI 2024, ACM MM 2024, IEEE Access, and arXiv. The following table summarises the findings.

Paper	Core Approach	Key Limitation	Gap Relevant to Our Work

Yildiz et al. (2023)	Audio-based violence detection	No visual/spatial awareness	Audio-only; not applicable to pre-entry visual exploration
Chen et al. (AI-ITA 2025)	UCVL + large model benchmark	Static surveillance only	No mobility; no onboard inference capability
Liu et al. (2024)	VLM scene anomaly recognition	High compute cost	GPU-dependent; cloud-required; not embeddable
Yang et al. (CVPR 2024)	Text-prompt weakly supervised VAD	Fixed camera assumption	No exploration or dynamic repositioning
Zanella et al. (CVPR 2024)	LLM training-free VAD	High latency for robotics	Unsuitable for embedded real-time use
Wu et al. (AAAI 2024)	VadCLIP vision-language VAD	Cloud inference required	No offline or mobile deployment path
He et al. (arXiv 2024)	MobileMamba — lightweight SSM	Classification only; not robotic	Not integrated in any first-response robotic system
Sun et al. (ACM MM 2024)	TDSD text-driven VAD	No mobility integration	No SLAM component; no first-response focus
Zou et al. (arXiv 2025)	Fine-grained VLM prompting	High resource requirement	Not deployable on embedded hardware

2.4 Identified Research Gaps

Based on the literature survey, the following four research gaps are formally identified. These gaps together constitute the novelty space that this project occupies.

- No existing system combines SLAM-based autonomous exploration with lightweight onboard anomaly detection in a single deployable unit. All surveyed systems assume static camera infrastructure.
- Fine-tuned State-Space Models such as Mobile Mamba have never been applied to indoor crime scene Region of Interest detection. The existing application is image classification, not robotic perception.
- No surveyed system adopts a first-response framing — the goal of providing pre-entry situational awareness for live, hostile environments rather than retrospective accuracy.
- No system provides hash-verified, encrypted forensic logging entirely on a mobile embedded platform without cloud dependency. Forensic integrity on embedded hardware is an open problem.

3 Execution Plan

3.1 Subdomains and Learning Areas

The project requires competency across five distinct technical subdomains. Each subdomain has been mapped to specific system components and learning resources.

Subdomain	System Components Addressed	Resources Used	Planned Learning Toward Complete Solution
Autonomous Robotics & SLAM	Navigation module, frontier exploration, occupancy grid mapping	Thrun et al. – *Probabilistic Robotics*; ROS tutorials; Hector SLAM paper	Study of 3D SLAM methods and implementation of additional Z-axis Degree of Freedom enabling transition between perpendicular planes (ground-to-wall mobility)
Lightweight AI / SSMs	Mobile Mamba inference, layered fine-tuning, ONNX export	MobileMamba arXiv:2411.15941; Mamba paper (Gu & Dao 2023); PyTorch training documentation	Edge deployment optimization through quantization, real-time inference acceleration, and memory-efficient execution on embedded platforms
Anomaly Detection	ROI detection pipeline, anomaly scoring, dataset curation	9 surveyed research papers; UCF Crime Dataset documentation	Temporal anomaly tracking, lightweight video understanding, and multi-modal anomaly fusion for improved detection robustness
Embedded Systems / Hardware	Raspberry Pi , camera integration, IMU sensing, motor driver control	RPi datasheet; TensorFlow Lite ARM benchmark guide; MPU-6050 I2C guide	Sensor fusion architecture, power-efficient embedded autonomy pipeline, and integration with real-time navigation stack
Security & Forensics	SHA-256 hashing, AES-256 encryption, structured logging pipeline	NIST SP 800-101; PyCryptodome docs; Python hashlib documentation	Tamper-evident storage design, secure mission logging workflow, and post-mission encrypted synchronization framework

3.2 Implementation Phases

The project is structured across four implementation phases spanning the current and subsequent semesters.

Phase 1 — Literature Review and Architecture Design (Completed)

- Surveyed nine research papers across CVPR, AAAI, ACM MM, IEEE Access, and arXiv.
- Identified four formal research gaps and mapped novelty of this project against existing literature.
- Defined the full system architecture covering Sense, Navigate, Trigger, Perceive, Decide, Store, and Sync stages.
- Selected Mobile Mamba as the core AI model and defined the Layered Fine-Tuning strategy (frozen backbone $\rightarrow$ progressive unfreeze of `blocks3` with differential learning rates).
- Finalised hardware component list: Raspberry Pi , Pi Camera, MPU-6050 IMU, HC-SR04, L298N motor driver.
- Designed the forensic logging schema and security pipeline (SHA-256 + AES-256).

Phase 2 — Model Fine-Tuning and System Integration

- Collect and annotate indoor anomaly frames from UCF Crime Dataset and custom indoor recordings.

- Fine-tune MobileMamba using Layered fine tuning with PyTorch on curated dataset.
- Export fine-tuned model to ONNX and TFLite format for Raspberry Pi deployment.
- Integrate inference pipeline with the robot's camera input and event trigger module.
- Connect SLAM navigation module with the ROI detection pipeline.
- Unit-test each pipeline stage independently before full integration.

Phase 3 — Testing, Optimisation, and Validation

- Benchmark inference latency on Raspberry Pi; validate $<$ 150ms target.
- Calibrate ROI anomaly threshold using validation set from indoor dataset.
- Measure battery consumption during simulated missions; verify return-at-20% protocol.
- Validate SLAM mapping accuracy in a controlled indoor test environment.
- Verify SHA-256 integrity across stored ROI frames to confirm forensic chain of custody.
- Execute full end-to-end autonomous indoor mission run.

Phase 4 — Demonstration and Documentation

- Record and submit full autonomous mission demonstration video.
- Generate and validate post-mission ROI report output.
- Submit final report with all sections completed per 23CSE399 guidelines.
- Commit all code and design documentation to GitHub repository for design proof.

4 Fundamentals Learned — Study Methodology

4.1 Overview

The following section documents all learning completed during the current semester in preparation for system implementation. Each domain maps directly to one or more components of the project architecture, and the learning sources include research papers, textbooks, official documentation, and online lecture series.

4.2 Autonomous Robotics and SLAM

The team studied SLAM (Simultaneous Localisation and Mapping) as the foundational concept for the navigation module. SLAM allows a robot to construct a real-time map of an unknown environment while simultaneously estimating its own position within that map, using only onboard sensor data. The frontier-based exploration strategy was studied in detail: the robot identifies the boundaries between known-free space and unexplored space (frontiers), selects the nearest frontier as its next navigation goal, plans a path to it, and updates the map continuously as it moves. This strategy guarantees systematic, high-coverage exploration without redundant traversal.

Primary resources: Thrun, Burgard, and Fox — *Probabilistic Robotics* (textbook); ROS Navigation Stack tutorials (Robotics Back-end channel); Hector SLAM paper (IEEE 2011); Frontier-based exploration algorithm from Yamauchi (1997), referenced in multiple surveyed papers.

4.3 Lightweight AI — State-Space Models and Mobile Mamba

This project fine-tunes Mobile Mamba on indoor anomaly data using a **Layered Fine-Tuning** strategy. In the initial phase (epochs 1–15), the entire backbone is frozen and only the 6-class classification head is trained, allowing the head to stabilise without corrupting pre-trained weights. From epoch 16 onward, the final backbone stage (`blocks3`) and the head are unfrozen with a backbone learning rate of 5×10^{-6} and a head learning rate of 3×10^{-4}. Early layers (`blocks1`, `blocks2`, and `patch_embed`) remain frozen throughout, preserving the fundamental spatial feature extraction learned during ImageNet pre-training. This approach was preferred over LoRA/PEFT because MobileMamba-B1 is already compact, making direct block-level control more efficient than adding rank-decomposition adapters.

Primary resources: Mamba paper (Gu and Dao, 2023); MobileMamba paper (He et al., 2024); PyTorch training documentation; MobileMamba trainer.py implementation;; ONNX Runtime documentation; TFLite ARM Cortex-A53 benchmark guide (TensorFlow official documentation).

4.4 Video Anomaly Detection — Dataset and Literature

To ground the project in the broader field of Video Anomaly Detection (VAD), nine papers were formally studied and tabulated (see Section 2.3). The UCF Crime Dataset was reviewed in detail: it contains 1,900 real-world CCTV videos across 13 crime categories including Assault, Robbery, Shooting, and Arson, annotated at the video level with weak supervision. While the dataset originates from static cameras, the visual patterns of displaced objects, unusual postures, and scene disruptions are transferable to the indoor exploration context. The team also reviewed the UCVL benchmark (Chen et al., AIITA 2025) as a supplementary reference for large-model performance baselines.

4.5 Embedded Systems and Hardware

The hardware selection process required studying several components in detail. The Raspberry Pi was benchmarked for TFLite inference using the ARM Cortex-A53 performance profiles available in TensorFlow's official documentation. The MPU-6050 IMU was studied for its I2C communication protocol and integration with GPIO on the Pi . The HC-SR04 ultrasonic sensor, L298N motor driver, and Li-Po battery management were studied through manufacturer datasheets and community tutorials for RPi-based robotics projects. Battery ADC monitoring for threshold-based return-home was reviewed through GPIO ADC library documentation.

4.6 Security and Forensic Integrity

Two security standards were studied for the data handling pipeline. SHA-256 hashing (via Python's `hashlib` library) is used to generate a unique fingerprint for each stored ROI frame, ensuring that no data is tampered with between collection and review. AES-256 encryption (via PyCryptodome) is applied before any data is transmitted over Wi-Fi to a command server. The NIST SP 800-101 standard on mobile forensics was reviewed to understand evidence integrity requirements. The team also designed a structured JSON logging schema for per-mission reports, covering: timestamp, SLAM coordinates, anomaly confidence score, and frame file reference with its SHA-256 hash.

4.7 Simulation Environment — CoppeliaSim EDU

Before hardware deployment, the system architecture is validated in simulation using CoppeliaSim EDU. The simulator provides a physics-based robotics testing environment supporting differential-drive kinematics, sensor emulation, and external controller integration through Python and ROS interfaces. It enables testing of navigation logic, frontier-based exploration behaviour, and structured mission execution before transferring algorithms to embedded hardware.

The simulation environment is used to model indoor exploration scenarios, validate occupancy-grid-based movement decisions, and test ROI-triggered anomaly detection workflows integrated with the Mobile Mamba inference pipeline. Sensor behaviour such as camera perception and IMU-based orientation estimation is approximated within the simulator to ensure safe validation prior to real-world deployment.

Primary resources: CoppeliaSim EDU official documentation; ROS–CoppeliaSim interface tutorials; Python remote API integration guides for simulation-based robotics experimentation.

5 System Architecture and Design

5.0.1 Architecture Design Rationale

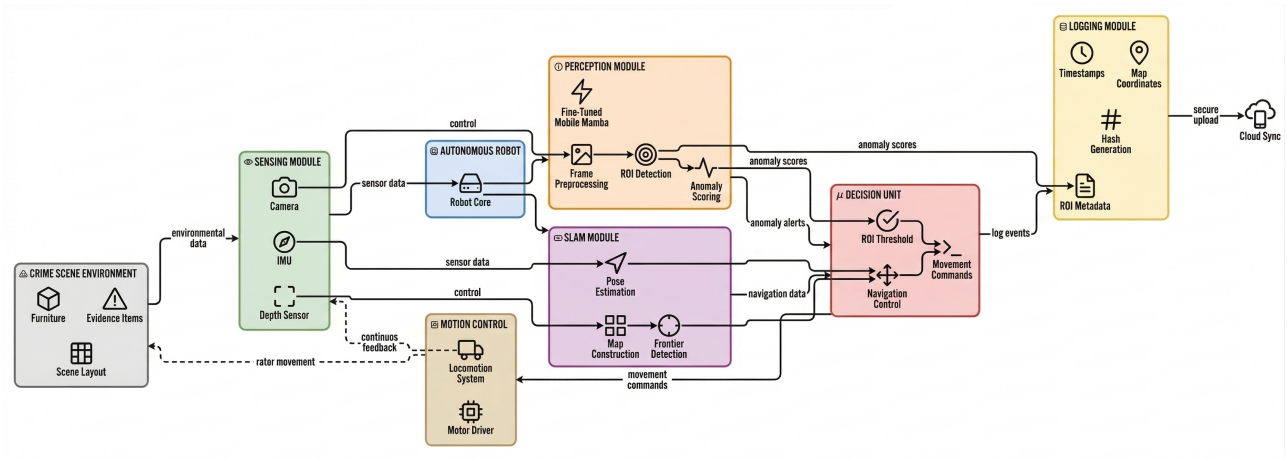


Figure 1: System Architecture

Sensing Module: The sensing module collects visual and inertial data required for localisation and environmental understanding. Camera frames provide visual input for anomaly detection, while IMU measurements support orientation estimation during SLAM updates. Ultrasonic depth sensing improves short-range obstacle awareness.

SLAM Module: The SLAM module constructs an occupancy grid representation of the environment while simultaneously estimating the robot pose. Frontier-based exploration ensures systematic coverage by directing the robot toward unexplored regions instead of random motion.

Event-Trigger Mechanism: An event-trigger mechanism activates the perception pipeline only when required. This reduces unnecessary inference operations and allows the system to operate within the computational limits of the Raspberry Pi platform.

Perception Module: The perception module uses the fine-tuned Mobile Mamba model to evaluate candidate frames and assign anomaly confidence scores. Only frames exceeding the detection threshold are retained as Regions of Interest (ROI).

Decision Module: The decision module determines whether detected anomalies should be logged and whether navigation behaviour should be modified based on detection confidence.

Logging Subsystem: The logging subsystem ensures forensic integrity by storing ROI metadata together with SHA-256 hashes. This prevents post-capture modification of evidence.

Synchronisation Module: Finally, the synchronisation module encrypts stored data using AES-256 before opportunistic wireless transmission to a remote command interface when connectivity becomes available.

5.1 End-to-End Pipeline

The system architecture is structured as a linear pipeline of seven stages. Each stage is a discrete, testable module with defined inputs and outputs.

Stage	Module	Input / Output	Key Technologies
Sense	Sensing Module	Camera frames, IMU data, depth readings → fused sensor stream	Pi Camera v2, MPU-6050, HC-SR04
Navigate	SLAM + Frontier	Sensor stream → occupancy grid, robot pose, frontier list	Hector SLAM / GMapping, frontier algorithm
Trigger	Event Detector	Occupancy updates → perception activation signal	Threshold logic on sensor delta
Perceive	Mobile Mamba	Camera frame → anomaly confidence score [0,1]	Fine-tuned MobileMamba, TFLite, ONNX Runtime
Decide	Decision Unit	Confidence score → ROI flag, movement command	Threshold comparator, navigation controller
Store	Logging Module	ROI flag + metadata → hashed, buffered local log	SHA-256 (hashlib), structured JSON
Sync	Cloud Sync	Local log → AES-256 encrypted cloud upload (opportunistic)	AES-256 (PyCryptodome), Wi-Fi socket

5.2 Hardware Platform

The robot is built on a mobile chassis with four DC motors driven by an L298N motor driver. All computation is performed locally on a Raspberry Pi IV (quad-core ARM Cortex-A53 at 1GHz, 512MB RAM). The visual sensor is a Raspberry Pi Camera Module v2 (8MP Sony IMX219) connected via the CSI interface. An MPU-6050 IMU provides gyroscope and accelerometer data for SLAM pose estimation. An HC-SR04 ultrasonic sensor provides depth and obstacle distance information. The system is powered by a 3000mAh Li-Po battery with continuous ADC monitoring.

The Raspberry Pi IV was selected over heavier alternatives such as the Jetson Nano or Raspberry Pi 4 specifically to minimise chassis dimensions, weight, and power consumption — critical constraints for a mobile, battery-constrained first-response robot. The trade-off is limited compute headroom, which is addressed by the use of TFLite quantised inference and event-driven (rather than continuous) perception.

5.3 Why Mobile Mamba?

The choice of Mobile Mamba over Transformer-based models is driven by the computational constraints of the Raspberry Pi Transformer models use self-attention with $O(n^2)$ complexity, requiring GPU memory that is unavailable on the target hardware. Mobile Mamba's selective state-scan mechanism achieves O(n) linear complexity, a fixed-size memory footprint, and has been benchmarked to achieve real-time inference on ARM processors at low quantisation levels. The fine-tuned model, exported via ONNX or TFLite, is the only model in our survey that can meet the $<$ 150ms latency target on a CPU-only embedded platform.

5.4 SLAM Navigation Design

The navigation system uses SLAM to simultaneously build an occupancy grid of the environment and estimate the robot's position within it. The occupancy grid divides the floor plan into cells labelled as free, occupied, or unknown. Frontier detection identifies cells on the boundary between known-free space and unknown space. The robot always targets the nearest frontier, plans a collision-free path to it using a path planner such as

Dijkstra or A*, navigates there, and updates the map. This continues until no unexplored frontiers remain or the battery threshold is reached, at which point the robot initiates its return to the starting position.

5.5 ROI Detection Pipeline

The perception module operates in an event-driven mode rather than processing every camera frame continuously. A sensor trigger fires when the robot enters a new region or during a deliberate scan phase. The captured frame is preprocessed (resized, normalised) and passed to the fine-tuned Mobile Mamba model, which outputs a binary ROI classification and a floating-point anomaly confidence score between 0 and 1. If the score exceeds a calibrated threshold, the frame is flagged as an ROI, and its metadata — timestamp, SLAM coordinates, score, and file path — is written to the local JSON log together with its SHA-256 hash. Non-ROI frames are discarded to conserve storage, with a policy that non-ROI frames are dropped first when storage exceeds 80% capacity.

6 Summary of Works Completed — Semester 1

6.1 Research and Analysis

- Surveyed nine research papers covering video anomaly detection, vision-language models, LLM-based surveillance, and lightweight SSMs.
- Produced a structured gap analysis table identifying four formal research gaps (Section 2.4).
- Reviewed UCF Crime Dataset documentation, class distribution, and annotation methodology.
- Benchmarked Mobile Mamba against Transformer baselines in terms of complexity, latency, and embedded suitability.

6.2 Architecture and Design

- Designed the complete seven-stage system pipeline (Sense through Sync) with defined inter-module interfaces.
- Documented the SLAM frontier exploration strategy and occupancy grid management approach.
- Designed the event-driven ROI detection pipeline with a six-step flow from sensor trigger to forensic log.
- Created the system architecture diagram showing all modules, data flows, and hardware connections.

6.3 Model,Dataset and Robot

- Selected Mobile Mamba (arXiv:2411.15941) as the base model with justification against Transformer and VLM alternatives.
- Extended the model from image-based inference to video classification for temporal anomaly understanding.
- Implemented initial training on UCF Crime Dataset using five classes: Assault, Robbery, Shooting, Fighting, and Abuse.
- Implemented Layered Fine-Tuning strategy: head-only training for epochs 1–15, followed by partial unfreeze of `blocks3` with differential learning rates (backbone 5×10^{-6}, head 3×10^{-4}).
- Achieved approximately 50% baseline accuracy on 5-class UCF Crime classification as a working prototype for further fine-tuning.
- Planned integration of curated indoor anomaly dataset including displaced objects, unusual arrangements, fallen human postures, abandoned items, and stains.

- Identified ONNX and TFLite as deployment formats for ARM CPU inference.
- Implemented basic autonomous navigation using SLAM with A* path planning for structured exploration.

6.4 Hardware Specification

- Finalised hardware component list with individual justification for each component.
- Documented Raspberry Pi as the onboard processor with ARM Cortex-A53 TFLite benchmark validation.
- Defined battery management protocol: continuous ADC monitoring, return-home at 20% threshold.
- Specified mobile chassis type, motor driver wiring, and GPIO communication plan.

6.5 Security and Forensic Pipeline

- Designed SHA-256 hashing protocol for per-frame forensic integrity.
- Designed AES-256 encryption pipeline for cloud synchronisation.
- Specified structured JSON logging schema: timestamp, coordinates, anomaly score, frame reference, hash value.
- Reviewed NIST SP 800-101 for evidence integrity compliance requirements.
- Reviewed MISRA C:2012 embedded coding guidelines for safe GPIO and sensor interfacing on the Raspberry Pi platform.
- Referenced ISO/IEC 25010 software quality model (reliability, maintainability, performance efficiency) as the framework for evaluating system design trade-offs.

6.6 Documentation and Reporting

- Submitted Panel Review 1 report (February 2026) and received panel feedback.
- Addressed all Panel Review 1 feedback items in this submission (see Section 7 of this report).
- Prepared Panel Review 2 presentation covering all required CO1, CO2, CO3 components.
- Initialised GitHub repository with architecture diagrams and design documentation for design proof.

7 Panel Review 1 Feedback — Response and Actions Taken

The following questions were raised by the panel during Panel Review 1 (February 2026). Each item is addressed with the clarification made and the action taken in this submission.

Panel Question	Clarification	Action Taken in PR2
Where will the robot be deployed?	Post-crime, pre-entry hostage scenarios inside buildings where human entry is unsafe.	Dedicated scenario slide and Section 1.3 of this report.
Before crime or after crime?	After the crime / incident has occurred — robot enters before human responders, not before the attacker.	Clarified in all scenario descriptions throughout report and presentation.

What is your dataset?	UCF Crime Dataset (primary) + custom indoor anomaly frames for fine-tuning.	Dataset documented in Sections 4.4 and 3.1; referenced in architecture slides.
Multiple robots?	Single robot for now. Swarm robotics identified as future improvement.	Future improvements formally documented in Section 8 .
Why robot instead of cameras?	Cameras can be hacked or destroyed. Robot operates where fixed infrastructure is compromised.	Addressed in Problem Context (Slide 2, Section 2.1), and use case justification.

8 Plans for Oncoming Semesters

8.1 Semester 2 Plan — System Integration and Simulation

- **Model Optimisation:** Convert the fine-tuned MobileMamba model to TFLite INT8 format and benchmark inference latency on Raspberry Pi. Target: $<$150ms.
- **Simulation Platform Development:** Build a mobile robotic system in CoppeliaSim with appropriate Degree of Freedom (DoF) modeling.
- **SLAM Integration:** Implement and validate SLAM-based exploration (GMapping / Hector SLAM equivalent) in simulation.
- **Path Planning:** Integrate Path planning algorithm for structured navigation within the simulated environment.
- **Pipeline Integration:** Combine navigation, perception (Mobile Mamba), and event-triggered inference into a unified simulation pipeline.
- **Security Pipeline Validation:** Test SHA-256 hashing and AES-256 encryption on simulated ROI data.

8.2 Semester 3 Plan — Validation and Demonstration

- **Simulation-Based Mission Execution:** Perform complete autonomous exploration missions in simulation with ROI detection and logging.
- **Latency Benchmarking:** Evaluate inference latency and system performance under embedded constraints.
- **ROI Calibration:** Tune anomaly thresholds using dataset + simulation outputs.
- **Model Deployment Validation:** Demonstrate Mobile Mamba running on Raspberry Pi with real-time feasibility.
- **Final Deliverables:** Complete system demonstration including:
  - Fully integrated simulation (navigation + perception + decision)
  - Optimized lightweight model ($<$150ms inference)
  - Structured ROI logging and security pipeline
  - Demonstration video and documentation

9 Limitations and Future Improvements

9.1 Known Limitations

- The robot cannot enter if no physical access point (door, window, passage) is available. This is an inherent physical constraint acknowledged as a deployment prerequisite.
- Mid-mission battery depletion may prevent the robot from returning. Mitigated by the 20% return threshold, but sudden motor stalls or rough terrain may accelerate consumption.
- If the robot becomes physically stuck — wedged under furniture or caught in debris — it cannot be retrieved remotely. A physical retrieval tether is a low-tech mitigation considered for future versions.
- No real-time communication is implemented in the current phase. Data is retrieved only when the robot physically returns or after post-mission Wi-Fi sync. This is acceptable for intelligence gathering but not for real-time operator awareness.
- The inference pipeline targets $<$ 150ms on the Raspberry Pi , but this target has not yet been validated on hardware. Validation is planned for Phase 2.

9.2 Future Improvements (Planned Beyond Current Scope)

- **Swarm Robotics:** Multiple coordinated robots for large-scale or multi-floor environments. One robot can retrieve or assist another if stuck. Parallel exploration reduces total mission time. This will be formally proposed in the next review phase.
- **Advanced Communication Module:** Adaptive frequency hopping to counter RF jammers; mesh networking between swarm units; LoRa or 5G fallback for connectivity beyond Wi-Fi range. The communication module is deliberately left out of the current phase and deferred to future development.
- **3D SLAM and Stair Navigation:** Upgrade from 2D occupancy mapping to 3D volumetric mapping (e.g., OctoMap). Enables multi-floor exploration in buildings with accessible stairways.
- **Wireless Charging Stations:** Pre-positioned charging pads at entry points allow mid-mission recharging, eliminating the battery depletion limitation for extended missions.
- **Operator Dashboard:** Real-time status display (SLAM map, battery level, ROIs flagged) transmitted to a tablet at the command post when connectivity is available.
- **On-Device Scene Summary:** A quantised small language model (e.g., Phi-2 or TinyLlama at INT4) generating a plain-English summary of the mission findings from ROI metadata — entirely onboard without cloud inference.

References

[1] A. M. Yildiz, P. D. Barua, S. Dogan, M. Baygin, T. Tuncer, C. P. Ooi, H. Fujita, and U. R. Acharya, "A novel tree pattern-based violence detection model using audio signals," *Expert Systems with Applications*, vol. 224, p. 120031, 2023.

[2] H. Chen, D. Yi, M. Cao, C. Huang, G. Zhu, and J. Wang, "UCVL: A benchmark for crime surveillance video analysis with large models," in *Proc. 5th Int. Conf. on Artificial Intelligence and Industrial Technology Applications (AIITA)*, 2025, pp. 2051–2057.

[3] C. Liu, X. Long, Y. Li, C. Chen, F. Gu, S. Yuan, and C. Zhang, "Improving anomaly scene recognition with large vision-language models," 2024.

[4] Z. Yang, J. Liu, and P. Wu, "Text prompt with normality guidance for weakly supervised video anomaly detection," in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[5] H. He *et al.*, "MobileMamba: Lightweight Multi-Receptive Visual Mamba Network," arXiv preprint arXiv:2411.15941, 2024.

[6] P. Wu, X. Zhou, G. Pang, L. Zhou, Q. Yan, P. Wang, and Y. Zhang, "VadCLIP: Adapting vision-language models for weakly supervised video anomaly detection," in *Proc. AAAI Conf. on Artificial Intelligence*, 2024, pp. 6074–6082.

[7] S. Sun, J. Hua, J. Feng, D. Wei, B. Lai, and X. Gong, "TDSD: Text-driven scene-decoupled weakly supervised video anomaly detection," in *Proc. ACM Int. Conf. on Multimedia (MM)*, 2024.

[8] L. Zanella, W. Menapace, M. Mancini, Y. Wang, and E. Ricci, "Harnessing large language models for training-free video anomaly detection," in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[9] S. Zou, X. Tian, L. Wesemann, F. Waschkowski, Z. Yang, and J. Zhang, "Unlocking vision-language models for video anomaly detection via fine-grained prompting," arXiv preprint arXiv:2510.02155, 2025.